Week 4, Day 3: Redis, Caching Patterns and Binary Search on the Answer

Prepared for Upendra Kumar Sharma | 4 hours | Main topic: Redis data structures, cache-aside, write-through, TTL, eviction, stampede, penetration, avalanche | DSA: Binary search on the answer, median of two sorted arrays

Today's Schedule

- **0:00 to 0:40** Study what Redis is and its data structures (sections 1 and 2).
- **0:40 to 1:20** Study caching patterns and invalidation (section 3).
- **1:20 to 2:00** Study cache failure modes and their fixes (section 4).
- **2:00 to 2:30** Study TTL, eviction, persistence and high availability (section 5).
- **2:30 to 2:50** Study Redis in Spring Boot (section 6).
- **2:50 to 4:10** DSA: solve the 3 problems in section 8. The first two take about 20 minutes each, and the median problem is hard, so allow about 40 minutes.
- **4:10 to 4:40** Answer the interview questions aloud (section 9) and add 10 flashcards.
- **Optional (90 min):** do the hands-on experiments in section 7 and add caching to your project.

1. What Redis Is

- Redis is an **in-memory data structure store**. It keeps data in RAM, so reads and writes take well under a millisecond. It is used as a **cache**, a session store, a rate limiter, a leaderboard, a lock service, a lightweight message broker and more.
- **Single-threaded command execution:** commands run one at a time on the main thread (network I/O can use extra threads in recent versions). So **each individual command is atomic**, and no two commands interleave. The flip side is that **one slow command blocks everyone**.
- **Persistence (optional): RDB** snapshots at intervals, and **AOF** (append-only file) that logs every write, with fsync policies such as everysec. When Redis is only a cache, many teams turn persistence off or keep it light.
- **Scaling and availability:** replication (asynchronous), **Sentinel** for automatic failover, and **Redis Cluster** for sharding data across nodes using **16,384 hash slots**.
- **Licensing note:** Redis changed its license in recent years, and **Valkey** is a compatible open-source fork. Managed services such as AWS ElastiCache support these engines. In interviews, say "Redis or a compatible engine" and keep the focus on concepts.

- **Redis is not your source of truth** when used as a cache. Design so that losing the whole cache costs performance, not correctness.

2. Data Structures and Commands

Strings (the basic type; also integers and binary data)

- GET, SET, MGET. **SET key value EX 300 NX** sets with a 300-second expiry only if the key does not exist.
- **INCR and INCRBY** are atomic counters. Used for rate limiting, view counts and id generation.

Hashes: a map of fields inside one key (HSET, HGET, HGETALL, HINCRBY). Good for objects, since you can update a single field without rewriting everything.

Lists: a linked list (LPUSH, RPUSH, LPOP, BRPOP, LRANGE). Used for simple queues and recent-items lists.

Sets: unique, unordered members (SADD, SISMEMBER, SINTER, SUNION). Used for tags, unique visitors and relationships.

Sorted sets (ZSET): unique members with a **score**, kept ordered (ZADD, ZRANGE, ZREVRANGE, ZRANK, ZRANGEBYSCORE, ZREMRANGEBYSCORE). Used for **leaderboards**, **sliding-window rate limiting** (Day 4), and delayed job queues using the time as a score.

Others to name: HyperLogLog (approximate unique counts in tiny memory), **bitmaps** (daily active flags), **geospatial indexes**, **Streams** (a log with consumer groups, similar in spirit to Kafka), and **Pub/Sub** (fire and forget, with no history, so a disconnected subscriber misses messages).

Using Redis safely

- **Key naming:** use clear namespaces with a version, for example product:v1:123. Versioned keys let you change the cached format without clearing everything.
- **Avoid big keys and slow commands.** HGETALL, SMEMBERS, LRANGE 0 -1 and similar commands on huge collections block the server. Never use **KEYS** in production (it scans everything). Use **SCAN**, which is incremental.
- **Atomicity beyond one command: MULTI and EXEC** run a group of commands together (there is no rollback of already executed commands), **WATCH** adds optimistic checking, and **Lua scripts** (EVAL) run several steps atomically on the server. **Pipelining** sends many commands in one round trip to cut network latency.
- **Hot keys:** one key receiving a very large share of traffic overloads one node. Local caching or key replication helps.

3. Caching Patterns

Cache-aside (lazy loading), the most common pattern

- **Read:** check the cache. On a hit, return it. On a miss, load from the database, **store the result in the cache with a TTL**, and return it.
- **Write:** update the database, then **delete** (invalidate) the cache key. The next read repopulates it. Deleting is safer than updating the cache, because it avoids writing a value that a concurrent request has already made stale.
- **Pros:** caches only what is actually requested, and the application still works if the cache is empty or down. **Cons:** the first request after a miss is slow, and there is a window where the cache can be stale.

Read-through: the cache layer itself loads data from the database on a miss. Same idea as cache-aside, but the loading logic lives in the cache library or provider.

Write-through: every write goes to the cache **and** the database in the same operation. The cache is always up to date, but writes are slower, and you may cache data that is never read.

Write-behind (write-back): write to the cache first and flush to the database **later, asynchronously**. Very fast writes and batching, but a cache failure before the flush **loses data**, so only use it where some loss is acceptable and you have strong safeguards.

Refresh-ahead: refresh popular entries **before** they expire, so users rarely hit a miss. Good for hot, predictable data.

A race to know with cache-aside

1. Request A misses and reads the old value from the database.
2. Request B updates the database and deletes the cache key.
3. Request A then writes its **old** value into the cache.
4. The cache is now stale until the TTL ends.

Mitigations: always set a **TTL** as a safety net, use short TTLs for data where staleness matters, version your keys or values, or use a delayed second delete. For data that must be exact (money, stock to reserve), **do not decide from the cache**. Read the database.

Invalidation strategies

- **TTL only:** simplest, with bounded staleness.
- **Explicit delete on write:** fresh data, but every write path must remember to do it.
- **Event-driven:** publish an event when data changes (through your outbox and Kafka, or CDC) so every cache layer invalidates itself. This fits your microservice design.

- Remember the saying that cache invalidation is one of the hard problems. Pick the simplest strategy that meets the freshness requirement, and state that requirement.

4. Cache Failure Modes and Fixes

Cache stampede (thundering herd or dogpile)

- A hot key **expires**, and hundreds of concurrent requests all miss and all hit the database to rebuild it.
- **Fixes:**
  - **Single-flight or a per-key lock:** only one request rebuilds the value while the others wait briefly or serve slightly stale data. In one process use a local lock. Across instances use a short Redis lock (SET NX with a short expiry).
  - **TTL jitter:** add a random amount to each TTL so keys do not expire together.
  - **Stale-while-revalidate (logical expiry):** store the value with a soft expiry time. After it, serve the stale value and refresh it in the background.
  - **Refresh-ahead** and **pre-warming** for known hot keys.

Cache penetration

- Requests for keys that **do not exist** (typos, scanners, attacks) always miss and always reach the database.
- **Fixes: cache the "not found" result** for a short time (negative caching), validate input before looking anything up, and for very large key spaces use a **Bloom filter** that quickly says definitely not present.

Cache avalanche

- Many keys expire at the same moment, or the **cache is down**, and the database is flooded.
- **Fixes:** TTL jitter, **high availability** for Redis (replicas, Sentinel or Cluster), a small **local cache** in front, protecting the database with **rate limiting and a circuit breaker** (Week 3, Day 3), and graceful degradation (serve less data rather than fail).

Hot key: a single very popular key. Add a **local in-process cache** (for example Caffeine) with a short TTL, or store several copies under suffixed keys and pick one at random.

Design rule: the application must **work when Redis is slow or down.** Use short client timeouts (a few hundred milliseconds), catch Redis exceptions, and fall back to the database, protected as above. A cache outage should never become a full outage.

5. TTL, Eviction, Persistence and High Availability

Expiry (TTL)

- Set with EXPIRE, SET ... EX, or PEXPIRE, and read with TTL. Without a TTL, a key lives forever.
- Redis removes expired keys in two ways: **lazily** when the key is accessed, and by **periodically sampling** keys that have an expiry. So expired keys may use memory for a short while.

Eviction when memory is full (maxmemory and maxmemory-policy)

- **noeviction:** the default. Writes fail with an error when memory is full.
- **allkeys-lru:** evict the least recently used key among all keys. The usual choice for a **pure cache**.
- **allkeys-lfu:** evict the least frequently used key. Better when a few keys are consistently popular.
- **volatile-lru, volatile-lfu, volatile-ttl:** consider only keys that have a TTL. Useful when the instance holds a mix of cache data (with TTL) and data that must stay (without TTL).
- **allkeys-random and volatile-random** also exist. LRU and LFU in Redis are **approximate** (sampling based), which is fast and good enough.

Persistence

- **RDB:** periodic snapshots. Compact and quick to restart, but you lose data since the last snapshot.
- **AOF:** logs each write, with fsync every second (at most about one second of loss) or always (slow). Larger files, and it is rewritten in the background.
- **For a cache:** persistence is optional. For data you cannot rebuild, use AOF and replicas, and think carefully about whether Redis is the right primary store.

High availability and scale

- **Replication is asynchronous,** so a failover can lose the most recent writes.
- **Sentinel** monitors a primary and its replicas and promotes a replica on failure.
- **Cluster** shards keys across several primaries by hash slot, and each primary has replicas. Multi-key commands and Lua scripts only work when all keys are in the same slot. Use **hash tags** (for example {user:42}:profile and {user:42}:cart) to put related keys together.
- **Managed services** (ElastiCache, Azure Cache, Memorystore) handle most of this.

What to monitor: hit ratio (keyspace_hits compared with keyspace_misses in INFO stats), evicted_keys, used_memory compared with maxmemory, connected clients, command latency and the slow log, replication lag, and the number of keys.

6. Redis in Spring Boot

Configuration (Spring Boot 3 uses the spring.data.redis prefix)

```
spring:
  data:
    redis:
      host: redis
      port: 6379
      timeout: 500ms
      connect-timeout: 500ms
```

Cache-aside by hand with StringRedisTemplate (so you understand what the annotations hide)

```
@Service
public class StockCache {
    private static final String NULL_MARKER = "__NULL__";
    private final StringRedisTemplate redis;
    private final ObjectMapper mapper;
    private final StockRepository repository;
    // constructor omitted

    public StockView get(String sku) {
        String key = "stock:v1:" + sku;
        try {
            String cached = redis.opsForValue().get(key);
            if (cached != null) {
                return NULL_MARKER.equals(cached) ? null : fromJson(cached);
            }
        } catch (RedisConnectionFailureException ex) {
            return loadFromDatabase(sku);              // cache is down:
degrade, do not fail
        }

        StockView view = loadFromDatabase(sku);
        Duration ttl = (view == null)
                ? Duration.ofSeconds(30)                                  //
negative caching
                : Duration.ofSeconds(300 +
ThreadLocalRandom.current().nextInt(60));   // TTL jitter
```

```
        try {
            redis.opsForValue().set(key, view == null ? NULL_MARKER :
toJson(view), ttl);
        } catch (RuntimeException ignored) { /* a failed cache write must not
fail the request */ }
        return view;
    }

    public void evict(String sku) {
        redis.delete("stock:v1:" + sku);       // call this after the
database update commits
    }
}
```

- **Serialization:** prefer JSON (readable, and tolerant of added fields) over Java serialization. Include a version in the key so a format change cannot break old entries.
- **Where to evict:** after the database transaction **commits**, for example in a @TransactionalEventListener(AFTER_COMMIT) or from an event consumer, so a rolled-back change does not clear the cache for nothing.
- **Important for your project:** cache stock for **display** (GET /api/inventory/{sku}). **Never use the cached value to decide a reservation.** Reservations must use the atomic database update.
- **Tomorrow:** Spring Cache annotations (@Cacheable, @CacheEvict) with Redis, plus rate limiting and locks.

7. Optional Hands-On (90 Minutes)

Use the redis container from your Compose file, and open the shell with docker compose exec redis redis-cli.

1. **Data structure drills.** Try SET with EX and NX, INCR, HSET and HGETALL, LPUSH and LRANGE, SADD and SINTER, ZADD with ZREVRANGE WITHSCORES, and PFADD with PFCOUNT. Use TTL to watch an expiry count down.
2. **Expiry.** Set a key with a 5-second TTL and read it every second. Note when it disappears.
3. **Eviction.** Run CONFIG SET maxmemory 5mb and CONFIG SET maxmemory-policy allkeys-lru. Write many keys with a small script and watch evicted_keys in INFO stats. Then switch to noeviction and see the out-of-memory error.
4. **Cache-aside in the project.** Add the StockCache above to inventory-service for the GET endpoint. Measure response times with and without the cache, and read the hit ratio from INFO stats.

5. **Stampede.** Set a short TTL, then send 100 parallel requests right after expiry and count the database queries (use SQL logging). Add a per-key lock and TTL jitter, and repeat.
6. **Penetration.** Request 1,000 random unknown skus and count database hits. Add negative caching and compare.
7. **Cache down.** Stop the Redis container while sending traffic. Confirm the API keeps working through the database, with response times visible in your logs, and that the timeout setting keeps requests from hanging.
8. **Invalidation.** Update stock through the admin endpoint and confirm the next read shows the new value. Try changing the database without evicting, and note how long staleness lasts.

Record your numbers (latency, hit ratio, database queries). Concrete before-and-after figures are exactly what a strong interview answer needs.

8. DSA: Binary Search on the Answer (3 Problems)

The pattern: sometimes you do not search an array, you search the **range of possible answers**. If you can write a function feasible(x) that answers "does x work?", and feasibility is **monotonic** (once it works for x, it works for every larger x), you can binary search for the smallest x that works.

```
lo = smallest possible answer, hi = largest possible answer
while lo < hi:
    mid = lo + (hi - lo) / 2
    if feasible(mid): hi = mid        # mid works, try smaller
    else:             lo = mid + 1    # mid fails, need larger
return lo
```

Problem 1: Koko Eating Bananas

- **The problem:** piles of bananas, h hours. Each hour she eats up to k bananas from one pile. Find the smallest k that finishes all piles within h hours.
- **Idea:** k ranges from 1 to the largest pile. feasible(k) means the total hours, the sum of ceiling(pile / k), is at most h. Larger k never takes more hours, so it is monotonic.
- **Complexity:** O(n log m), where m is the largest pile.
- **Edge cases:** h equal to the number of piles (k must be the largest pile), a single pile, very large piles (use long arithmetic).

```
public int minEatingSpeed(int[] piles, int h) {
    int lo = 1, hi = 0;
    for (int p : piles) hi = Math.max(hi, p);
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (canFinish(piles, mid, h)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}

private boolean canFinish(int[] piles, int k, int h) {
    long hours = 0;
    for (int p : piles) hours += ((long) p + k - 1) / k;     // ceiling
division
    return hours <= h;
}
```

Problem 2: Capacity To Ship Packages Within D Days

- **The problem:** packages with weights must be shipped **in the given order**, loaded onto a ship with a fixed capacity, within D days. Find the smallest capacity.
- **Idea:** the capacity is at least the heaviest package and at most the total weight. feasible(c) greedily loads packages each day until the next one would exceed c, and counts the days. If the days needed are at most D, the capacity works.
- **Complexity:** O(n log(sum of weights)).
- **Edge cases:** one package, D equal to 1 (the answer is the total weight), D equal to the number of packages (the answer is the heaviest).

```
public int shipWithinDays(int[] weights, int days) {
    int lo = 0, hi = 0;
    for (int w : weights) {
        lo = Math.max(lo, w);
        hi += w;
    }
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (daysNeeded(weights, mid) <= days) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}

private int daysNeeded(int[] weights, int capacity) {
```

```
    int days = 1, load = 0;
    for (int w : weights) {
        if (load + w > capacity) {
            days++;
            load = 0;
        }
        load += w;
    }
    return days;
}
```

Related problems to recognize: Split Array Largest Sum, Minimum Number of Days to Make Bouquets, and "minimize the maximum" or "maximize the minimum" questions in general.

Problem 3: Median of Two Sorted Arrays (hard)

- **Idea:** binary search the **partition point** in the **smaller** array. A partition of both arrays splits all elements into a left half and a right half, with the left half holding half of the elements (rounded up). The partition is correct when every element on the left is less than or equal to every element on the right, which means a[i-1] <= b[j] and b[j-1] <= a[i]. If a[i-1] is too large, move the partition left. Otherwise move it right.
- **Median:** with an odd total, the larger of the two left-side maximums. With an even total, the average of the left-side maximum and the right-side minimum.
- **Complexity:** O(log(min(m, n))) time, O(1) space.
- **Edge cases:** one array empty, all elements of one array smaller than all of the other, partitions at the ends (use minus and plus infinity sentinels), an even or odd total.
- Allow extra time. It is the hardest problem of the week, and explaining the partition idea clearly matters as much as the code. A simpler O(m + n) merge-based answer is a fine starting point.

```
public double findMedianSortedArrays(int[] a, int[] b) {
    if (a.length > b.length) return findMedianSortedArrays(b, a);    //
search the smaller array
    int m = a.length, n = b.length;
    int lo = 0, hi = m, half = (m + n + 1) / 2;

    while (lo <= hi) {
        int i = lo + (hi - lo) / 2;          // elements taken from a
        int j = half - i;                    // elements taken from b

        int aLeft  = (i == 0) ? Integer.MIN_VALUE : a[i - 1];
        int aRight = (i == m) ? Integer.MAX_VALUE : a[i];
        int bLeft  = (j == 0) ? Integer.MIN_VALUE : b[j - 1];
```

```
        int bRight = (j == n) ? Integer.MAX_VALUE : b[j];

        if (aLeft <= bRight && bLeft <= aRight) {
            if ((m + n) % 2 == 1) return Math.max(aLeft, bLeft);
            return (Math.max(aLeft, bLeft) + (long) Math.min(aRight, bRight))
/ 2.0;
        } else if (aLeft > bRight) {
            hi = i - 1;
        } else {
            lo = i + 1;
        }
    }
    throw new IllegalArgumentException("Input arrays are not sorted");
}
```

9. Interview Questions to Answer Aloud

1. What is Redis, and why is it fast? What does single-threaded command execution mean for atomicity and for slow commands?
2. Which Redis data structures do you know, and what is each used for? When would you use a sorted set?
3. Explain cache-aside. Why delete the key on write instead of updating it?
4. Compare cache-aside, read-through, write-through and write-behind. What are the risks of write-behind?
5. Describe a race that can leave stale data in a cache-aside setup, and how you would limit the damage.
6. What is a cache stampede? Give at least three ways to prevent it.
7. What is cache penetration, and how do negative caching and a Bloom filter help?
8. What is a cache avalanche, and how do you prevent it?
9. How do TTL and eviction policies work? Which policy would you choose for a pure cache, and why?
10. What is the difference between RDB and AOF? Do you need persistence for a cache?
11. How do Redis replication, Sentinel and Cluster differ? What are hash slots and hash tags?
12. Why should you never use the KEYS command in production?
13. How should your service behave if Redis is down or slow?
14. Which data would you never serve from a cache when making a decision, and why? (Think about stock reservation and balances.)

10. End-of-Day Checklist

- [ ] I can describe Redis's execution model and name the main data structures with a use for each.
- [ ] I can explain cache-aside, write-through and write-behind, and the stale data race.
- [ ] I can explain stampede, penetration and avalanche, with fixes for each.
- [ ] I can explain TTL, eviction policies, persistence options and Sentinel versus Cluster.
- [ ] I wrote the cache-aside code with TTL jitter, negative caching and a fallback when Redis is down.
- [ ] I solved the two binary-search-on-the-answer problems and can state the feasibility pattern.
- [ ] I worked through the median problem and can explain the partition idea.
- [ ] (Optional) My inventory read endpoint is cached, and I recorded latency, hit ratio and stampede results.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Week 4, Day 4): Redis for rate limiting and distributed locks, Spring Cache with Redis (@Cacheable, @CacheEvict), plus trie problems.